

that this function will receive a parameter, the slug of the article. We need to take this slug, and use it to fetch the article with that slug from the database. From the ‘Django Models’ chapter we know how to fetch a single item from the database:

- `Article.objects.get(slug=article _ slug)`

We’ll use this in our view function to get the correct article, fill in details from this article into an HTML string, then return this code in an [HttpResponse](#) and Bob’s your uncle! Although given the demographic our our readership your uncle’s probably Sanjay.

Here is the code for our view:

- ```
def article _ detail(request, slug):
 article = Article.objects.get(slug=slug)
 html = """
 <html>
 <head><title>{title}</title></head>
 <body>
 <h1>{title}</h1>
 <p>Author: {author}</p>
 {content}
 </body>
 </html>
 """
 response = html.format(
 title=article.title,
 author=article.author.username,
 category=article.category.name,
 content=article.content)
 return HttpResponse(response)
```

You will also need to map this view to the correct URL in the `urls.py` file. The correct entry mapped to this view will look like the following:

- ```
url(r'^articles/([\w-]+)$', views.article _ detail,
```
- ```
 name='articles-details')
```

You’ll note that we’ve added another bit to our URL configuration, a name. While it is not necessary to supply a name for your URL patterns, supplying one makes it easy to reverse a URL match, that is, to get a URL for an article given its ID or slug.

Now visit <http://localhost:8000/articles/test-article-1> and you will be greeted by a bare-bones page that shows the article title, author, category and content.